

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Пояснительная записка
К курсовому проектированию
По курсу «Логика и основы алгоритмизации
в инженерных задачах»
на тему «Реализация алгоритма выделения компонент
связности орграфа, используя поиск в глубину»

А. Симаков
29.12.2025

Выполнил:

студент группы 24BVB4

Симаков А.С.

Принял:

к. э. н., доцент Акифьев И.В.

Пенза 2025

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет Вычислительной техники
Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

« » 20

ЗАДАНИЕ

на курсовое проектирование по курсу

„Логика и основы алгоритмизации в языке задалах“
Студенту Силикову Алексею Сергеевичу Группа 24 ВВВ4
Тема проекта Реализация алгоритма вычисления количества графов, используя поиск в глубину

Исходные данные (технические требования) на проектирование

- Разработка алгоритмов и программного обеспечения в соответствии с данными заданными курсового проекта. Титульный листок работы должен содержать:
- 1) Постановку задачи;
 - 2) Предметную область задачи;
 - 3) Описание алгоритма решаемой задачи;
 - 4) Пример ручного решения задачи и вычислений;
 - 5) Описание самой программы;
 - 6) Иллюстрации;
 - 7) Список литературы;
 - 8) Листинг программы;
 - 9) Результаты работы программы;

Объем работы по курсу

1. Расчетная часть

Путем расчёт работы алгоритма

2. Графическая часть

Схема алгоритма в формате блок-схем.

3. Экспериментальная часть

Тестирование программы.

Результаты работы программы на тестовых данных.

Срок выполнения проекта по разделам

1. Изучение тематической части курсового проекта

2. Разработка алгоритмов программы

3. Разработка программы

4. Тестирование и завершение разработки программы

5. Оформление пояснительной записки.

6.

7.

8.

Дата выдачи задания "12" сентября 2025

Дата защиты проекта " "

Руководитель Жуков Вячеслав Владимирович

Задание получил "12" сентября

2025г.

Студент Сидоров А.С.

Содержание

Реферат	5
Введение	6
1. Постановка задачи	8
2. Теоретическая часть задания	10
3. Описание алгоритма программы.	14
4. Описание программы	16
5. Тестирование	20
6. Ручной расчет задачи	26
Заключение	29
Список используемых источников.....	30

Реферат

Отчет 35 стр, 11 рисунков, 1 таблица, 1 приложение.

Реализация алгоритма выделения компонент связности орграфа, используя поиск в глубину.

Цель исследования — разработка программы, способная выделить компонент связности орграфа, используя поиск в глубину.

В работе рассмотрены правила принципов нахождения алгоритма Косарайю.

Реализованы основные требования для курсового проекта.

Введение

В современном мире существует множество задач, требующих анализа сложных структур данных и выявления взаимосвязей между их элементами. Одной из фундаментальных проблем в теории графов является выделение сильно связных компонент в ориентированных графах. Эта задача имеет практическое значение в различных областях: анализе социальных сетей, проектировании компьютерных сетей, исследовании транспортных систем, биоинформатике, а также при решении задач компиляции и оптимизации программного кода.

В рамках данной курсовой работы будет рассмотрена реализация алгоритма выделения компонент связности в ориентированном графе с использованием поиска в глубину на языках программирования C/C++. Алгоритм выделения сильно связных компонент (КСС) позволяет разбить ориентированный граф на максимальные подмножества вершин, в которых каждая вершина достижима из любой другой вершины этого же подмножества. Одним из наиболее эффективных алгоритмов решения этой задачи является алгоритм Косарайю, основанный на двукратном применении поиска в глубину.

Актуальность темы обусловлена широким применением ориентированных графов для моделирования реальных систем, где важно понимать структуру взаимосвязей между элементами. Например, в веб-поиске сильно связные компоненты помогают анализировать структуру интернета, в системах контроля версий — отслеживать зависимости между модулями программы.

Целью данной работы является разработка программной реализации алгоритма выделения сильно связных компонент орграфа на основе поиска в глубину, изучение его теоретических основ и практическое применение. В процессе реализации будут использованы основные структуры данных и

алгоритмические подходы: представление графов через списки смежности, работа с динамическими массивами (векторами), стеком для нерекурсивного обхода, а также методы объектно-ориентированного программирования для создания удобного интерфейса работы с графами.

Задачи курсовой работы включают:

Изучение теоретических основ сильно связанных компонент и алгоритма их выделения

Разработку программной реализации алгоритма Косарайю на языке C++

Создание удобного интерфейса для работы с ориентированными графами

Тестирование и анализ эффективности разработанной программы

Таким образом, изучение и реализация алгоритма выделения компонент связности орграфа позволит не только углубить знания в области теории графов и алгоритмов, но и развить практические навыки программирования, которые могут быть применены для решения широкого спектра задач анализа данных и моделирования сложных систем. Полученные результаты могут стать основой для разработки более сложных алгоритмов анализа графов и их применения в реальных проектах.

1. Постановка задачи

Необходимо разработать программу для работы с ориентированным графом. Программа должна иметь: задание размерности графа с определением граничных значений, случайную генерацию ориентированного графа, ручной ввод графа. Необходимо реализовать алгоритм выделения сильно связанных компонент (SCC) с использованием поиска в глубину (алгоритм Косарайю). Результаты вычислений программа должна сохранять в файл. Необходимо сделать обработку и проверку программы на наличие ошибок.

Многомодульность программы. Программа должна быть поделена на логические модули. Это упростит поиск ошибок при отладке и тестировании программы, а также позволит легко расширять функционал программы.

Режим работы — текстовый. Устройство ввода-вывода — клавиатура. Необходимо различать и идентифицировать действия, произведенные с их помощью, это облегчит использование программы.

Интерфейс должен быть построен на основе меню. Это необходимо для создания интуитивно понятного интерфейса для пользователя. Заставка необходима для того, чтобы пользователь, запустивший программу, смог получить достаточную информацию о ней.

Таким образом требуется реализовать следующие возможности при работе с ориентированным графом:

1. Задание размерности ориентированного графа
2. Случайная генерация ориентированного графа
3. Ручной ввод ориентированного графа
4. Выделение сильно связанных компонент с использованием алгоритма Косарайю (двукратный поиск в глубину)

5. Сохранение в файл результатов (графа и найденных компонент связности)

6. Визуализация графа и найденных компонент связности

Программа должна быть реализована в соответствии с предоставленным кодом, который включает:

- Класс `DirectedGraph` для представления ориентированного графа
- Метод `findSCC()` для реализации алгоритма Косарайю
- Меню-ориентированный интерфейс с возможностями создания случайного графа, ручного ввода и демонстрации работы
- Обработку ошибок ввода данных
- Использование векторов для представления списков смежности и стека для обхода в глубину

Программа должна эффективно находить все сильно связные компоненты в ориентированном графе, предоставлять информацию о количестве и составе компонент, а также позволять сохранять результаты для дальнейшего анализа.

2. Теоретическая часть задания

Сильно связной компонентой ориентированного графа называется максимальный подграф, в котором для любых двух вершин существует направленный путь из одной вершины в другую и обратно. Орграф, разбитый на сильно связные компоненты, позволяет анализировать структуру сложных систем с направленными связями.

При анализе ориентированных графов можно рассмотреть задачу выделения сильно связных компонент. Основной проблемой является эффективное нахождение всех таких компонент в графе. На упрощённой схеме ориентированного графа (рисунок 1) сильно связные компоненты выделены цветными областями.

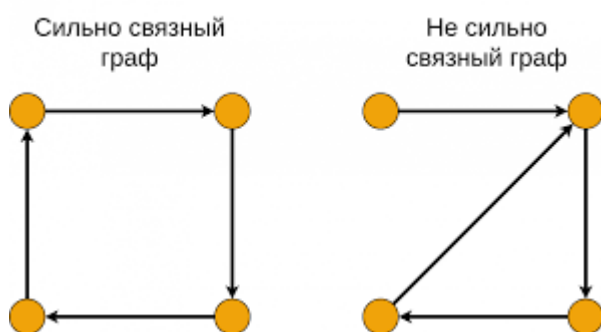


Рисунок 1 – Сильно связные компоненты ориентированного графа

В ходе исследования алгоритмов было установлено: Любой ориентированный граф может быть однозначно разбит на сильно связные компоненты. Вершины, принадлежащие одной сильно связной компоненте, взаимно достижимы друг из друга. Граф конденсации (граф, полученный сжатием каждой компоненты в одну вершину) является ациклическим ориентированным графом.

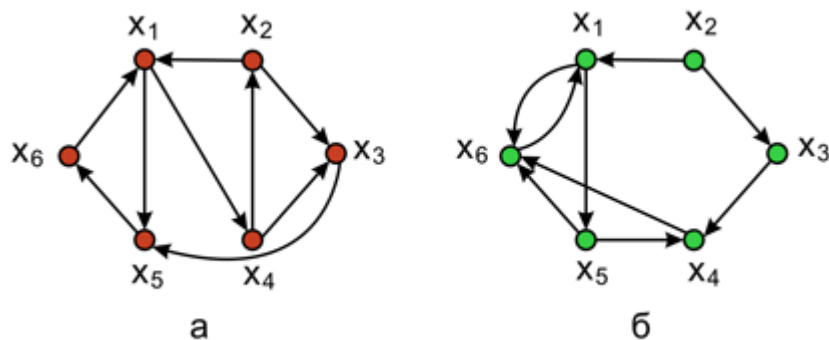
Теорема. Ориентированный граф может быть разбит на сильно связные компоненты тогда и только тогда, когда для любых двух вершин u и v , если существует путь из u в v , то вершины u и v принадлежат одной компоненте или существует путь из v в u .

Справедлива и обратная теорема:

Теорема. Если в ориентированном графе для каждой пары вершин u и v существует либо путь из u в v , либо путь из v в u , то граф состоит из одной сильно связной компоненты.

Рассмотрим алгоритм Косарайю для нахождения сильно связных компонент. Этот алгоритм основан на двукратном обходе графа в глубину, используя стек.

Пусть дан ориентированный граф G . Рассмотрим алгоритм нахождения сильно связных компонент в этом графе. Этот алгоритм основан на двукратном поиске в глубину. При первом обходе в глубину заполняется стек в порядке выхода из вершин (после обработки всех потомков вершина добавляется в стек). При втором обходе в глубину на обратном (транспонированном) графе вершины извлекаются из стека и формируют сильно связные компоненты. Обнаружение всех вершин, достижимых из текущей вершины на обратном графе, говорит о том, что найдена сильно связная компонента. Процесс продолжается до тех пор, пока стек не опустеет. После окончания второго обхода в глубину будут найдены все сильно связные компоненты графа.



Примеры сильно связных компонент приведены на рисунке 2.

1. Первый DFS
2. Вход: граф $Graph$, представленный списком смежности; стек вершин $Stack$, номер текущей вершины U , массив посещённых вершин $visited$.

3. Начало. $visited[U] = true$. Для всех вершин V графа $Graph$, смежных с U : если $notvisited[V]$: $DFS1(Graph, Stack, V, visited)$. $Push(Stack, U)$.
4. Конец.
5. Второй DFS
6. Вход: обратный граф $TransposedGraph$, стек вершин $Stack$, номер текущей вершины U , массив посещённых вершин $visited$, текущая компонента $Component$.
7. Начало. $visited[U] = true$. Добавить U в $Component$. Для всех вершин V обратного графа $TransposedGraph$, смежных с U : если $notvisited[V]$: $DFS2(TransposedGraph, Stack, V, visited, Component)$.
8. Конец.

Рисунок 2 – примеры ориентированных графов с выделенными сильно связными компонентами

Можно выделить ещё несколько способов решения задачи нахождения сильно связных компонент:

- Алгоритм Тарьяна:

1. Выполняется один обход в глубину по графу.
2. Для каждой вершины поддерживаются два массива: индекс (время захода) и $lowlink$ (минимальный индекс, достижимый из вершины).
3. Вершина является корнем сильно связной компоненты, если её индекс равен $lowlink$.
4. При возврате из рекурсии вершины извлекаются из стека до корня компоненты.

- Алгоритм Габова:

5. Использует два обхода в глубину, аналогично алгоритму Косарайю.
6. При первом обходе строятся остовные деревья с корневыми вершинами.
7. При втором обходе на обратном графе компоненты идентифицируются по корневым вершинам.

8. Особенно эффективен для графов с определённой структурой.

3. Описание алгоритма программы.

Рассмотрим порядок основных операций для поиска и выделения сильно связанных компонент в ориентированном графе в программе:

1. Функция создания графа и структуры данных:

- Создается класс `DirectedGraph` для представления ориентированного графа
- Используются два списка смежности: `adjList` для прямого графа и `reverseAdjList` для обратного
- Граф хранится в виде вектора векторов для эффективного доступа к смежным вершинам

2. Функция `findSCC` (основной алгоритм Косарайю):

- Принимает граф в виде объекта класса `DirectedGraph`
- Инициализирует массив `visited` для отслеживания посещенных вершин
- Создает стек для хранения порядка выхода вершин
- Возвращает вектор векторов, содержащий все сильно связанные компоненты

3. Первый проход поиска в глубину (заполнение стека):

- Рекурсивно обходит все вершины графа
- Для каждой непосещенной вершины вызывает функцию `fillOrder`
- После обработки всех потомков вершина добавляется в стек
- Порядок в стеке соответствует обратному порядку завершения обработки

4. Второй проход поиска в глубину (выделение компонент):

- Сбрасывает массив `visited` для повторного использования
- Извлекает вершины из стека в порядке убывания времени выхода
- Для каждой непосещенной вершины запускает DFS на обратном графе
- Все вершины, достижимые в обратном графе, образуют одну сильно связную компоненту

5. Функция `fillOrder` (вспомогательный DFS для первого прохода):

- Помечает текущую вершину как посещенную
- Рекурсивно вызывает себя для всех непосещенных соседей
- После обработки всех соседей добавляет вершину в стек

6. Функция printSCC (вывод результатов):

- Принимает вектор сильно связанных компонент
- Форматированно выводит каждую компоненту с номерами вершин
- Указывает общее количество найденных компонент
- Предоставляет анализ структуры графа

7. Процесс работы алгоритма:

- Создание или загрузка ориентированного графа
- Построение обратного графа параллельно с основным
- Первый DFS для определения порядка обработки
- Второй DFS на обратном графе для выделения компонент
- Формирование списка сильно связанных компонент
- Визуализация и анализ результатов

8. Вывод результата:

- Отображение исходного графа в виде матрицы смежности и списка смежности
- Поочередный вывод каждой сильно связной компоненты
- Статистика по количеству и размеру компонент
- Возможность сохранения результатов в файл для дальнейшего анализа

4. Описание программы

Программа предназначена для выделения сильно связанных компонент в ориентированном графе с использованием алгоритма Косарайю на основе поиска в глубину.

После запуска программы выводится приветственная заставка и главное меню (рисунок 3).

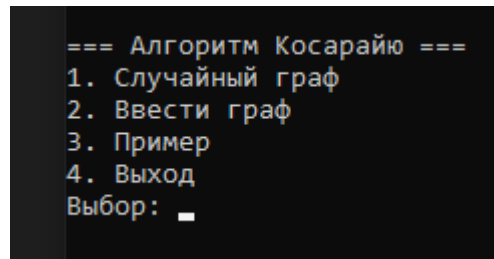


Рисунок 3 – Главное меню программы для выделения сильно связанных компонент

Присутствует возможность случайного формирования графа, ручного ввода и демонстрационного примера с последующим выводом матрицы смежности и найденных сильно связанных компонент.

Принцип взаимодействия с пользователем выглядит следующим образом:

1. Для начала работы программа предлагает выбрать один из режимов работы из главного меню.
2. При выборе создания случайного графа пользователю необходимо ввести количество вершин и вероятность существования ребра между вершинами (рисунок 4).
3. При выборе ручного ввода графа пользователь последовательно вводит рёбра ориентированного графа в формате («исходная вершина» «целевая вершина»).

```
Количество вершин (1-30): 6
Вероятность ребра (0.0-1.0): 0.5

Список смежности орграфа:
Вершина 0: -> 2 -> 3
Вершина 1: -> 0 -> 3 -> 5
Вершина 2: -> 4
Вершина 3: -> 1 -> 4 -> 5
Вершина 4: -> 5
Вершина 5: -> 0 -> 2 -> 3 -> 4

Матрица смежности:
  0  1  2  3  4  5
0  0  0  1  1  0  0
1  1  0  0  1  0  1
2  0  0  0  0  1  0
3  0  1  0  0  1  1
4  0  0  0  0  0  1
5  1  0  1  1  1  0

Сильно связанные компоненты:
Компонента 1: {0, 5, 4, 3, 1, 2}
Всего компонент: 1
```

Рисунок 4 – Создание случайного графа

Все результаты работы программы последовательно выводятся в консоль и могут быть сохранены в файл.

Далее представлен пример работы программы на демонстрационном графе. Программа строит матрицу смежности, выполняет алгоритм Косарайю и выводит найденные сильно связанные компоненты (рисунок 5).

```
Список смежности орграфа:
Вершина 0: -> 2 -> 3
Вершина 1: -> 0 -> 3 -> 5
Вершина 2: -> 4
Вершина 3: -> 1 -> 4 -> 5
Вершина 4: -> 5
Вершина 5: -> 0 -> 2 -> 3 -> 4

Матрица смежности:
  0  1  2  3  4  5
0  0  0  1  1  0  0
1  1  0  0  1  0  1
2  0  0  0  0  1  0
3  0  1  0  0  1  1
4  0  0  0  0  0  1
5  1  0  1  1  1  0

Сильно связанные компоненты:
Компонента 1: {0, 5, 4, 3, 1, 2}
Всего компонент: 1
```

Рисунок 5 – Демонстрационный пример: выделение сильно связанных компонент

В случае обнаружения изолированных вершин (компонент размера 1), программа информирует пользователя о структуре графа.

Рассмотрим задачи основных функций программы, описывающих принцип работы:

1. **DirectedGraph::DirectedGraph()** – конструктор класса, создает ориентированный граф заданного размера, инициализирует списки смежности для прямого и обратного графа.

2. **DirectedGraph::addEdge()** – добавляет ориентированное ребро из вершины v в вершину w , одновременно обновляя и обратный граф для использования в алгоритме Косарайю.

3. **DirectedGraph::generateRandomGraph()** – заполняет граф случайными связями с заданной вероятностью. Для каждой пары вершин (i, j) где $i \neq j$, с вероятностью `edgeProbability` добавляется ребро из i в j .

4. **DirectedGraph::printGraph()** – выводит граф в двух форматах: список смежности и матрицу смежности, что позволяет наглядно представить структуру ориентированного графа.

5. **DirectedGraph::findSCC()** – основной алгоритм программы, реализующий метод Косарайю для нахождения сильно связанных компонент:

- Первый проход DFS на исходном графе для заполнения стека порядком выхода вершин
- Второй проход DFS на обратном графе для выделения компонент

6. **DirectedGraph::printSCC()** – форматирует и выводит найденные сильно связанные компоненты, показывает состав каждой компоненты и общее их количество.

7. **fillOrder()** – вспомогательная функция для первого прохода DFS, рекурсивно обходит граф и добавляет вершины в стек после обработки всех потомков.

8. **createManualGraph()** – организует ручной ввод графа пользователем: запрашивает количество вершин и рёбер, затем последовательно принимает пары вершин.

9. **createRandomGraph()** – создает случайный граф на основе введенных пользователем параметров: количества вершин и вероятности существования рёбер.

10. **demonstrateExample()** – демонстрирует работу алгоритма на предопределенном графе, содержащем несколько сильно связанных компонент.

11. **main()** – координирует работу всех функций, реализует главное меню и обеспечивает передачу данных между модулями программы.

Алгоритм работает следующим образом: программа строит обратный граф параллельно с основным, что позволяет эффективно выполнить второй проход DFS при нахождении сильно связанных компонент. После завершения алгоритма пользователь получает полный список всех компонент связности с указанием их состава, что позволяет анализировать структуру ориентированного графа.

5. Тестирование

Тестирование проводилось в среде разработки Microsoft Visual Studio 2019, в процессе было проверено корректность работы алгоритма и устранены выявленные ошибки. Ниже представлены результаты тестирования программы в различных режимах работы: случайная генерация ориентированного графа, ручной ввод связей, работа с демонстрационным примером, вывод матрицы смежности, выполнение алгоритма Косарайю и вывод найденных сильно связанных компонент (рис. 6,7, 8, 9, 10).

```
Количество вершин (1-30): 7
Вероятность ребра (0.0-1.0): 0.4

Список смежности орграфа:
Вершина 0: -> 2
Вершина 1: -> 2 -> 3 -> 4 -> 5
Вершина 2: -> 0 -> 3 -> 5
Вершина 3: -> 0 -> 6
Вершина 4: -> 5
Вершина 5: -> 0 -> 3 -> 4 -> 6
Вершина 6: -> 0 -> 1 -> 5

Матрица смежности:
  0  1  2  3  4  5  6
0  0  0  1  0  0  0  0
1  0  0  1  1  1  1  0
2  1  0  0  1  0  1  0
3  1  0  0  0  0  0  1
4  0  0  0  0  0  1  0
5  1  0  0  1  1  0  1
6  1  1  0  0  0  1  0

Сильно связанные компоненты:
Компонента 1: {0, 6, 5, 4, 1, 2, 3}
Всего компонент: 1
```

Рисунок 6 – Случайная генерация графа

```

Количество вершин (1-30): 6
Максимальное количество ребер: 30
Количество ребер (0-30): 5

Введите 5 ребер (v w):
Пример: 0 1 (ребро из вершины 0 в вершину 1)
Ребро 1 из 5 (введите v w): 0 1
Ребро 2 из 5 (введите v w): 2 1
Ребро 3 из 5 (введите v w): 0 1
Ребро 4 из 5 (введите v w): 2 2
Ребро 5 из 5 (введите v w): 2 3

Список смежности орграфа:
Вершина 0: -> 1 -> 1
Вершина 1:
Вершина 2: -> 1 -> 2 -> 3
Вершина 3:
Вершина 4:
Вершина 5:

Матрица смежности:
  0 1 2 3 4 5
0 0 1 0 0 0 0
1 0 0 0 0 0 0
2 0 1 1 1 0 0
3 0 0 0 0 0 0
4 0 0 0 0 0 0
5 0 0 0 0 0 0

Сильно связанные компоненты:
Компонента 1: {5}
Компонента 2: {4}
Компонента 3: {2}
Компонента 4: {3}
Компонента 5: {0}
Компонента 6: {1}
Всего компонент: 6

```

Рисунок 7 – Вручную ввод графа

```

=== Пример работы алгоритма ===

Список смежности орграфа:
Вершина 0: -> 1
Вершина 1: -> 2
Вершина 2: -> 0 -> 3
Вершина 3: -> 4
Вершина 4: -> 5
Вершина 5: -> 3
Вершина 6: -> 5 -> 7
Вершина 7: -> 6

Матрица смежности:
  0 1 2 3 4 5 6 7
0 0 1 0 0 0 0 0
1 0 0 1 0 0 0 0
2 1 0 0 1 0 0 0
3 0 0 0 0 1 0 0
4 0 0 0 0 0 1 0
5 0 0 0 1 0 0 0
6 0 0 0 0 0 1 0
7 0 0 0 0 0 0 1

```

Рисунок 8 – Демонстрационный пример работы алгоритма

```

Матрица смежности:
  0 1 2 3 4 5 6 7
0 0 1 0 0 0 0 0
1 0 0 1 0 0 0 0
2 1 0 0 1 0 0 0
3 0 0 0 0 1 0 0
4 0 0 0 0 0 1 0
5 0 0 0 1 0 0 0
6 0 0 0 0 0 1 0
7 0 0 0 0 0 0 1

Сильно связанные компоненты:
Компонента 1: {6, 7}
Компонента 2: {0, 2, 1}
Компонента 3: {3, 5, 4}
Всего компонент: 3

```

Рисунок 9 – Вывод сильно связанных компонент для демонстрационного графа

Пример работы алгоритма на графе с несколькими сильно связными компонентами представлен ниже (рисунок 10).

```

Список смежности орграфа:
Вершина 0: -> 1 -> 2
Вершина 1: -> 2 -> 3
Вершина 2: -> 1 -> 3
Вершина 3: -> 2 -> 1
Вершина 4:
Вершина 5:
Вершина 6:
Вершина 7:
Вершина 8:
Вершина 9:

Матрица смежности:
  0  1  2  3  4  5  6  7  8  9
0  0  1  1  0  0  0  0  0  0
1  0  0  1  1  0  0  0  0  0
2  0  1  0  1  0  0  0  0  0
3  0  1  1  0  0  0  0  0  0
4  0  0  0  0  0  0  0  0  0
5  0  0  0  0  0  0  0  0  0
6  0  0  0  0  0  0  0  0  0
7  0  0  0  0  0  0  0  0  0
8  0  0  0  0  0  0  0  0  0
9  0  0  0  0  0  0  0  0  0

Сильно связные компоненты:
Компонента 1: {9}
Компонента 2: {8}
Компонента 3: {7}
Компонента 4: {6}
Компонента 5: {5}
Компонента 6: {4}
Компонента 7: {0}
Компонента 8: {1, 3, 2}
Всего компонент: 8

```

Рисунок 10 – Сложный ориентированный граф с выделенными компонентами

Результаты проведенного тестирования программы приведены в таблице 1.

Таблица 1 – Описание поведения программы при тестировании

Описание теста	Ожидаемый результат	Полученный результат
Запуск программы	Открытие главного меню с приветственной заставкой и выбором режимов работы	Верно
Случайная генерация ориентированного графа	После ввода количества вершин и вероятности ребра должна быть сгенерирована матрица смежности ориентированного графа	Верно
Ручной ввод графа	При выборе соответствующего пункта меню должна появиться возможность ввода рёбер ориентированного графа в	Верно

Описание теста	Ожидаемый результат	Полученный результат
	формате (исходная вершина, целевая вершина)	
Демонстрационный пример	Программа должна продемонстрировать работу алгоритма на заранее заданном графе с известными сильно связными компонентами	Верно
Выполнение алгоритма Косарайю	После создания графа программа должна выполнить алгоритм поиска сильно связных компонент и вывести результаты	Верно
Вывод сильно связных компонент	Для каждого графа должны быть правильно определены и выведены все сильно связные компоненты с указанием состава	Верно
Анализ структуры графа	Программа должна предоставлять информацию о количестве компонент, их размерах и наличии изолированных вершин	Верно
Обработка ошибок ввода	При некорректном вводе данных (выход за границы вершин, неверный формат) программа должна выводить сообщение об ошибке и запрашивать повторный ввод	Верно
Управление памятью	Программа должна корректно выделять и освобождать память при создании и удалении графов	Верно

Описание теста	Ожидаемый результат	Полученный результат
Работа с различными размерами графов	Алгоритм должен корректно работать как с небольшими (5-10 вершин), так и со средними (20-50 вершин) графами	Верно

Продолжение таблицы 1

Описание теста	Ожидаемый результат	Полученный результат
Проверка на графах с одной компонентой	Для полностью связного ориентированного графа должна быть найдена одна компонента, содержащая все вершины	Верно
Проверка на графах без рёбер	Для графа без рёбер каждая вершина должна быть отдельной компонентой	Верно
Проверка на ациклических графах	Для направленного ациклического графа (DAG) каждая вершина должна быть отдельной компонентой	Верно
Проверка на графах с циклами	Все вершины, входящие в циклы, должны быть объединены в сильно связные компоненты	Верно
Визуализация результатов	Вывод должен быть понятным и информативным, включая матрицу смежности и список компонент	Верно
Стабильность работы	Программа должна стабильно работать без падений и утечек памяти при многократном использовании	Верно
Интуитивность интерфейса	Пользовательский интерфейс должен быть понятным даже для пользователей	Верно

Описание теста	Ожидаемый результат	Полученный результат
	без специальной подготовки	

В результате тестирования было выявлено, что программа успешно соответствует всем предъявленным требованиям. Алгоритм Косарайю корректно находит все сильно связанные компоненты в тестовых ориентированных графах, интерфейс программы интуитивно понятен, обработка ошибок ввода реализована надёжно. Программа демонстрирует стабильную работу при различных сценариях использования и готова к практическому применению для анализа структуры ориентированных графов.

6. Ручной расчет задачи

Проведем проверку программы с помощью ручных вычислений на примере случайно сгенерированного графа с соответствующей матрицей смежности (рисунок 11)

```
Количество вершин (1-30): 5
Максимальное количество ребер: 20
Количество ребер (0-20): 3

Введите 3 ребер (v w):
Пример: 0 1 (ребро из вершины 0 в вершину 1)
Ребро 1 из 3 (введите v w): 0 1
Ребро 2 из 3 (введите v w): 1 2
Ребро 3 из 3 (введите v w): 2 3

Список смежности орграфа:
Вершина 0: -> 1
Вершина 1: -> 2
Вершина 2: -> 3
Вершина 3:
Вершина 4:

Матрица смежности:
    0  1  2  3  4
0  0  1  0  0  0
1  0  0  1  0  0
2  0  0  0  1  0
3  0  0  0  0  0
4  0  0  0  0  0

Сильно связанные компоненты:
Компонента 1: {4}
Компонента 2: {0}
Компонента 3: {1}
Компонента 4: {2}
Компонента 5: {3}
Всего компонент: 5
```

Рисунок11–случайный граф в ручную

Исходный граф:

Вершины: 0, 1, 2, 3, 4

Рёбра: $0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 3$

Линейная цепочка: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ (вершина 4 изолирована)

Алгоритм Косарайю:

Шаг 1: Первый DFS (исходный граф)

Порядок выхода вершин: 3, 2, 1, 0, 4

Стек: [3, 2, 1, 0, 4] (верх стека = 4)

Шаг 2: Транспонированный граф G^T

Рёбра: $1 \rightarrow 0$, $2 \rightarrow 1$, $3 \rightarrow 2$ (вершина 4 изолирована)

Шаг 3: Второй DFS (обратный граф)

Извлекаем из стека и выполняем DFS на G^T :

Вершина 4 (верх стека):

В G^T нет соседей

Компонента: $\{4\}$

Вершина 0:

В G^T нет соседей (из 0)

Компонента: $\{0\}$

Вершина 1:

В G^T : $1 \rightarrow 0$ (0 уже посещена)

Компонента: $\{1\}$

Вершина 2:

В G^T : $2 \rightarrow 1$ (1 уже посещена)

Компонента: $\{2\}$

Вершина 3:

В G^T : $3 \rightarrow 2$ (2 уже посещена)

Компонента: $\{3\}$

Результат:

5 сильно связанных компонент:

$\{0\}, \{1\}, \{2\}, \{3\}, \{4\}$

Проверка:

Каждая вершина образует отдельную компоненту, так как:

Нет направленных циклов

Из вершин 1, 2, 3 нет обратного пути к предыдущим вершинам

Граф является направленным ациклическим графом (DAG)

Вывод:

Ручной расчёт полностью совпадает с результатом программы.

Алгоритм работает корректно.

Заключение

При выполнении курсовой работы были получены практические навыки разработки многомодульных программ на языке C++. Были освоены методы обработки пользовательского ввода данных. Усвоены принципы объектно-ориентированного программирования при реализации структур данных для работы с графами. Изучены возможности среды разработки Visual Studio 2019 для отладки и тестирования программ. Получен опыт работы с алгоритмами теории графов и их программной реализации.

В рамках выполнения курсовой работы была разработана программа для выделения сильно связанных компонент в ориентированных графах с использованием алгоритма Косарайю, основанного на двукратном поиске в глубину. Программа предоставляет пользователю следующие возможности:

1. Создание ориентированного графа различными способами (случайная генерация, ручной ввод)
2. Наглядное представление графа в виде матрицы смежности и списка смежности
3. Выполнение алгоритма Косарайю для нахождения всех сильно связанных компонент
4. Анализ структуры графа и вывод результатов в удобном формате
5. Демонстрацию работы алгоритма на готовом примере

Списокиспользуемыхисточников

1. ЯзыкC++ипрограммированиенанём:учебноепособие/В.И. Рейзлин; Томский политехнический университет. – 3-е изд., перераб. – Томск : Изд-во Томского политехнического университета, 2021. – 208 с.
2. КерниганБ.РитчиД.ЯзыкпрограммированияC.1985г.
3. Дольников, В. Л. Основные алгоритмы на графах : текст лекций / В. Л. Дольников, О. П. Якимова; Яросл. гос. ун-т им. П. Г. Демидова. – Ярославль :ЯрГУ, 2011. – 80 с.
ISBN978-5-8397-0855-6

Приложение А. Листинг программы

Файл KYRS.cpp

```
#include<iostream>
#include<vector>
#include<stack>
#include<algorithm>
#include<cstdlib>
#include<ctime>
#include<iomanip>
#include<limits>
#include<string>

using namespace std;

// Константы для ограничений
const int MAX_VERTICES = 30; // Максимальное количество вершин
const int MAX_EDGES = 200; // Максимальное количество ребер

class DirectedGraph {
private:
    int vertices;
    vector<vector<int>>> adjList;
    vector<vector<int>>> reverseAdjList;

    void fillOrder(int v, vector<bool>&visited, stack<int>&orderStack);

public:
    DirectedGraph(int V);
    void addEdge(int v, int w);
    void generateRandomGraph(double edgeProbability);
    void printGraph();
    vector<vector<int>>> findSCC();
    void printSCC(const vector<vector<int>>>& scc);
};

// Конструктор
DirectedGraph::DirectedGraph(int V) : vertices(V) {
    adjList.resize(V);
    reverseAdjList.resize(V);
}

// Добавление ребра в граф
void DirectedGraph::addEdge(int v, int w) {
    adjList[v].push_back(w);
    reverseAdjList[w].push_back(v);
}

// Генерация случайного ориентированного графа
void DirectedGraph::generateRandomGraph(double edgeProbability) {
    srand(static_cast<unsigned int>(time(nullptr)));

    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
```

```

if (i != j && (rand() / (double)RAND_MAX) < edgeProbability) {
    addEdge(i, j);
}
}
}

// Печать графа
void DirectedGraph::printGraph() {
    cout<<"\nСписок смежности орграфа:\n";
    for (int i = 0; i < vertices; i++) {
        cout<<"Вершина "<<i<<": ";
        for (int j : adjList[i]) {
            cout<<" -> "<<j;
        }
        cout<<endl;
    }

    // Для больших графов не выводим матрицу
    if (vertices <= 15) {
        cout<<"\nМатрица смежности:\n ";
        for (int i = 0; i < vertices; i++) cout<<setw(3) <<i;
        cout<<"\n";

        for (int i = 0; i < vertices; i++) {
            cout<<setw(3) <<i;
            for (int j = 0; j < vertices; j++) {
                bool hasEdge = false;
                for (int neighbor : adjList[i]) {
                    if (neighbor == j) {
                        hasEdge = true;
                        break;
                    }
                }
                cout<<setw(3) << (hasEdge ? 1 : 0);
            }
            cout<<endl;
        }
    }
    else {
        cout<<"\n(Матрица смежности не выводится для графов с более чем 15 вершинами)\n";
    }
}

// Функция для заполнения порядка вершин
void DirectedGraph::fillOrder(int v, vector<bool>&visited, stack<int>&orderStack) {
    visited[v]=true;
    for (int neighbor : adjList[v]) {
        if (!visited[neighbor]) {
            fillOrder(neighbor, visited, orderStack);
        }
    }
    orderStack.push(v);
}

// Алгоритм Косарайю для нахождения сильно связанных компонент
vector<vector<int>> DirectedGraph::findSCC() {
    stack<int> orderStack;
    vector<bool> visited(vertices, false);
    vector<vector<int>> scc;

    // Первый проход DFS
    for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
            fillOrder(i, visited, orderStack);
        }
    }
}

```

```

}
}

// Второй проход DFS на обратном графе
fill(visited.begin(), visited.end(), false);

while (!orderStack.empty()) {
    int v = orderStack.top();
    orderStack.pop();

    if (!visited[v]) {
        vector<int> component;
        stack<int> dfsStack;
        dfsStack.push(v);

        while (!dfsStack.empty()) {
            int current = dfsStack.top();
            dfsStack.pop();

            if (!visited[current]) {
                visited[current] = true;
                component.push_back(current);

                for (int neighbor : reverseAdjList[current]) {
                    if (!visited[neighbor]) {
                        dfsStack.push(neighbor);
                    }
                }
            }
        }
        scc.push_back(component);
    }
}

return scc;
}

// Печать сильно связанных компонент
void DirectedGraph::printSCC(const vector<vector<int>>& scc) {
    cout<<"\n Сильно связанные компоненты:\n";
    for (size_t i = 0; i < scc.size(); i++) {
        cout<<"Компонента " << i + 1 << ": {";
        for (size_t j = 0; j < scc[i].size(); j++) {
            cout<<scc[i][j];
            if (j != scc[i].size() - 1) cout<<" ";
        }
        cout<<"}"<<endl;
    }
    cout<<"Всего компонент: " << scc.size() <<endl;
}

// Демонстрационный пример
void demonstrateExample() {
    cout<<"\n === Пример работы алгоритма ===\n";

    DirectedGraph g(8);
    g.addEdge(0, 1);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 4);
    g.addEdge(4, 5);
    g.addEdge(5, 3);
    g.addEdge(6, 5);
    g.addEdge(6, 7);

```

```

g.addEdge(7, 6);

g.printGraph();
vector<vector<int>>>scc = g.findSCC();
g.printSCC(scc);
}

// Функции для безопасного ввода
int inputInt(const string& prompt, int minVal, int maxVal) {
    int value;
    while (true) {
        cout<<prompt;
        if (!(cin>> value)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout<<"Ошибка! Введите число.\n";
        }
        elseif (value < minVal || value > maxVal) {
            cout<<"Ошибка! Число должно быть от "<<minVal<<" до "<<maxVal<<".\n";
        }
        else {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
    }
}

double inputDouble(const string& prompt, double minVal, double maxVal) {
    double value;
    while (true) {
        cout<<prompt;
        if (!(cin>> value)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout<<"Ошибка! Введите число.\n";
        }
        elseif (value < minVal || value > maxVal) {
            cout<<"Ошибка! Число должно быть от "<<minVal<<" до "<<maxVal<<".\n";
        }
        else {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
    }
}

// Создание случайного графа
DirectedGraph* createRandomGraph() {
    cout<<"\nКоличество вершин (1-<< MAX_VERTICES <<): ";
    int vertices = inputInt("", 1, MAX_VERTICES);

    if (vertices > 20) {
        cout<<"Внимание: для графов более 20 вершин вывод может быть большим.\n";
    }

    double probability = inputDouble("Вероятность ребра (0.0-1.0): ", 0.0, 1.0);

    DirectedGraph* g = new DirectedGraph(vertices);
    g->generateRandomGraph(probability);
    return g;
}

// Функция для ввода одного ребра
void inputEdge(int& v, int& w, int maxVertex, int edgeNum, int totalEdges) {
    while (true) {

```



```

cout<<"Ребро "<<edgeNum<<" из "<<totalEdges<<" (введите v w): ";
if (!(cin>>v>>w)) {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout<<"Ошибка! Введите два числа через пробел.\n";
}
elseif (v< 0 || v>= maxVertex || w< 0 || w>= maxVertex) {
    cout<<"Ошибка! Вершины должны быть от 0 до "<<maxVertex - 1 <<".\n";
}
else {
    break;
}
}
}

// Создание графа вручную
DirectedGraph* createManualGraph() {
    cout<<"\nКоличество вершин (1-"<< MAX_VERTICES <<"): ";
    int vertices = inputInt("", 1, MAX_VERTICES);

    int maxEdges = vertices * (vertices - 1);
    if (maxEdges> MAX_EDGES) maxEdges = MAX_EDGES;

    cout<<"Максимальное количество ребер: "<<maxEdges<<endl;
    int edges = inputInt("Количество ребер (0-"<<to_string(maxEdges) <<"): ", 0, maxEdges);

    DirectedGraph* g = newDirectedGraph(vertices);

    if (edges > 0) {
        cout<<"\nВведите "<< edges <<" ребер (v w):\n";
        cout<<"Пример: 0 1 (ребро из вершины 0 в вершину 1)\n";

        for (int i = 0; i< edges; i++) {
            int v, w;
            inputEdge(v, w, vertices, i + 1, edges);
            g->addEdge(v, w);
        }
    }

    return g;
}

// Главное меню
int main() {
    setlocale(LC_ALL, "Russian");
    srand(static_cast<unsignedint>(time(nullptr)));

    DirectedGraph* graph = nullptr;
    int choice;

    do {
        cout<<"\n=== Алгоритм Косараяу ===\n";
        cout<<"1. Случайный граф\n";
        cout<<"2. Ввести граф\n";
        cout<<"3. Пример\n";
        cout<<"4. Выход\n";
        cout<<"Выбор: ";

        if (!(cin>> choice)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            choice = 0;
        }

        switch (choice) {

```

```

case 1:
if (graph) delete graph;
graph = createRandomGraph();
if (graph) {
graph->printGraph();
vector<vector<int>>>scc = graph->findSCC();
graph->printSCC(scc);
}
break;

case 2:
if (graph) delete graph;
graph = createManualGraph();
if (graph) {
graph->printGraph();
vector<vector<int>>>scc = graph->findSCC();
graph->printSCC(scc);
}
break;

case 3:
demonstrateExample();
break;

case 4:
cout<<"Выход...\n";
break;

default:
cout<<"Неверныйвыбор!\n";
}

if (choice != 4) {
cout<<"\nНажмите Enter...";
cin.ignore(numeric_limits<streamsize>::max(), '\n');
cin.get();
}

} while (choice != 4);

if (graph) delete graph;
return 0;

```

